Train_Enjoyment_Model_J26_IT339.ipynb
Last Checkpoint: 2026-10-21 (autosaved)
Trusted
Python 3 (ipykernel) | Idle
File Edit View Insert Cell Kernel Widgets Help
Save Cell Cut Copy Run All Stop Restart Code
SLIIT Research Project J26-IT339 (Component 3)
Dataset: Tourism_Master_Val (1).csv (16,156 Revie 48,516 Samples)

🌍 TravelMate: AI-Based Adaptive & Explainable Itinerary Optimization

Component 3: Supervised Machine Learning Model for Tourist Personalized Enjoyment Prediction

Research Project ID: `J26-IT339`
Dataset: `C:\Users\User\Downloads\Tourism_Master_Validated (1).csv`
Algorithm: Random Forest Regressor (`sklearn.ensemble.RandomForestRegressor`) with ColumnTransformer Pipeline
Target Objective: Model MAE ≤ 0.70 on a 1-5 Enjoyment Rating Scale

---

Abstract & Methodology:

This notebook documents the end-to-end data preparation, feature engineering, 5-fold cross-validation, hyperparameter-tuned training, and comprehensive evaluation of the supervised tourist enjoyment prediction model for Sri Lanka tourism destinations.

The trained pipeline outputs personalized reward signals to the Reinforcement Learning Optimizer (DQN) to balance itinerary enjoyment, travel fatigue, admission expenditure, and temporal constraints.

In [1]:
```python
import sys
import os
import re
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

sns.set_theme(style="whitegrid", palette="muted")
print("Python Version:", sys.version.split()[0])
print("Libraries imported successfully!")
```

```
Python Version: 3.14.3
Libraries imported successfully!
```

1. Data Ingestion & Preprocessing

Loading the master validated TripAdvisor reviews dataset (`Tourism_Master_Validated (1).csv`). We extract:

1. Destination Category (Mapping raw location types to Cultural, Heritage, Wildlife, Nature, Adventure, Culinary, Beaches).
2. Country of Origin (Standardizing international tourist locations).
3. VADER Compound Sentiment from 1-5 star ratings and review polarity.
4. Estimated Foreigner Admission Tariff (USD).
5. Visit Duration / Dwell Time (minutes).
6. Tourist Archetype Stratification (`solo` , `couple` , `family`).

In [2]:
```
CSV_PATH = r"C:\Users\User\Downloads\Tourism_Master_Validated (1).csv"
df_raw = pd.read_csv(CSV_PATH)
print(f"Loaded {len(df_raw):,} validated TripAdvisor review records from: {CSV
print(f"Columns ({len(df_raw.columns)}): {list(df_raw.columns[:10])} ...")
df_raw[['Location_Name', 'Location_Type', 'User_Location', 'Rating', 'visit_du
```

```
Loaded 16,156 validated TripAdvisor review records from: C:\Users\User\Download
s\Tourism_Master_Validated (1).csv
Columns (42): ['review_id', 'Location_Name', 'destination_id', 'Located_City',
'Location', 'Location_Type', 'User_ID', 'User_Location', 'User_Locale', 'User_C
ontributions'] ...
```

Location_Name	Location_Type	User_Location	Rating	visit_duration_min
Arugam Bay	Beaches	Dunsborough, Australia	5	120
Arugam Bay	Beaches	Bendigo, Australia	4	120
Arugam Bay	Beaches	Melbourne, Australia	5	120
Arugam Bay	Beaches	Ericeira, Portugal	5	120
Arugam Bay	Beaches	Pistoia, Italy	5	120

2. Feature Engineering & Archetype Expansion

Each review is expanded across three distinct traveler archetypes (`solo` , `couple` , `family`) using destination suitability weights: $$\text{Target Enjoyment} = 0.60 \times \text{Archetype Destination Baseline} + 0.40 \times \text{Review Rating} + \Delta_{\text{match}}$$

In [3]:
```
# Display dataset summary after feature transformation
print(f"Total Model Training Samples: {len(df):,}")
print("Category Distribution:\n", df['category'].value_counts())
print("\nArchetype Counts:\n", df['traveler_type'].value_counts())
df.head(5)
```

```
Total Model Training Samples: 48,516
Category Distribution:
category
Heritage     9147
Cultural     9057
Wildlife     8931
Nature       7257
Beaches      6336
Culinary     5655
Adventure    2133

Archetype Counts:
traveler_type
solo      16172
couple    16172
family    16172
```

destination	category	traveler_type	origin_country	traveler_sentiment	foreigner_fee	typical_dwell	enjoyment_sc
Arugam Bay	Beaches	solo	Australia	0.88	0.0	120	3.886
Arugam Bay	Beaches	couple	Australia	0.88	0.0	120	4.775
Arugam Bay	Beaches	family	Australia	0.88	0.0	120	4.550
Arugam Bay	Beaches	solo	Australia	0.62	0.0	120	3.486
Arugam Bay	Beaches	couple	Australia	0.62	0.0	120	4.425

In [4]:
```
# Visualize Enjoyment Score by Archetype
plt.figure(figsize=(8, 5))
sns.boxplot(x="traveler_type", y="enjoyment_score", data=df, palette="Set2")
plt.title("Personalized Enjoyment Score Distribution by Archetype", fontsize=1
plt.xlabel("Tourist Archetype")
plt.ylabel("Calibrated Enjoyment Score (1-5)")
plt.show()
```



3. Pipeline Architecture & Train/Test Split

- **Training Set (80%):** 38,812 samples
- **Unseen Test Set (20%):** 9,704 samples
- **Preprocessor:** OneHotEncoder with `handle_unknown='ignore'` for categorical variables, `passthrough` for numerical features.
- **Estimator:** Random Forest Regressor with 120 decision trees, max depth of 16, and leaf regularizers to prevent overfitting.

In [5]:
```
features = ["category", "traveler_type", "origin_country", "vader_sentiment",
target = "enjoyment_score"

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, rand

categorical_features = ["category", "traveler_type", "origin_country"]
numerical_features = ["vader_sentiment", "foreigner_fee_usd", "typical_dwell_m

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), c
        ("num", "passthrough", numerical_features),
    ]
)

rf_regressor = RandomForestRegressor(
    n_estimators=120,
    max_depth=16,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1,
)

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", rf_regressor),
])

print(f"Training split: {len(X_train):,} samples | Test split: {len(X_test):,}
```

```
Training split: 38,812 samples | Test split: 9,704 samples
```

4. 5-Fold Cross-Validation Analysis

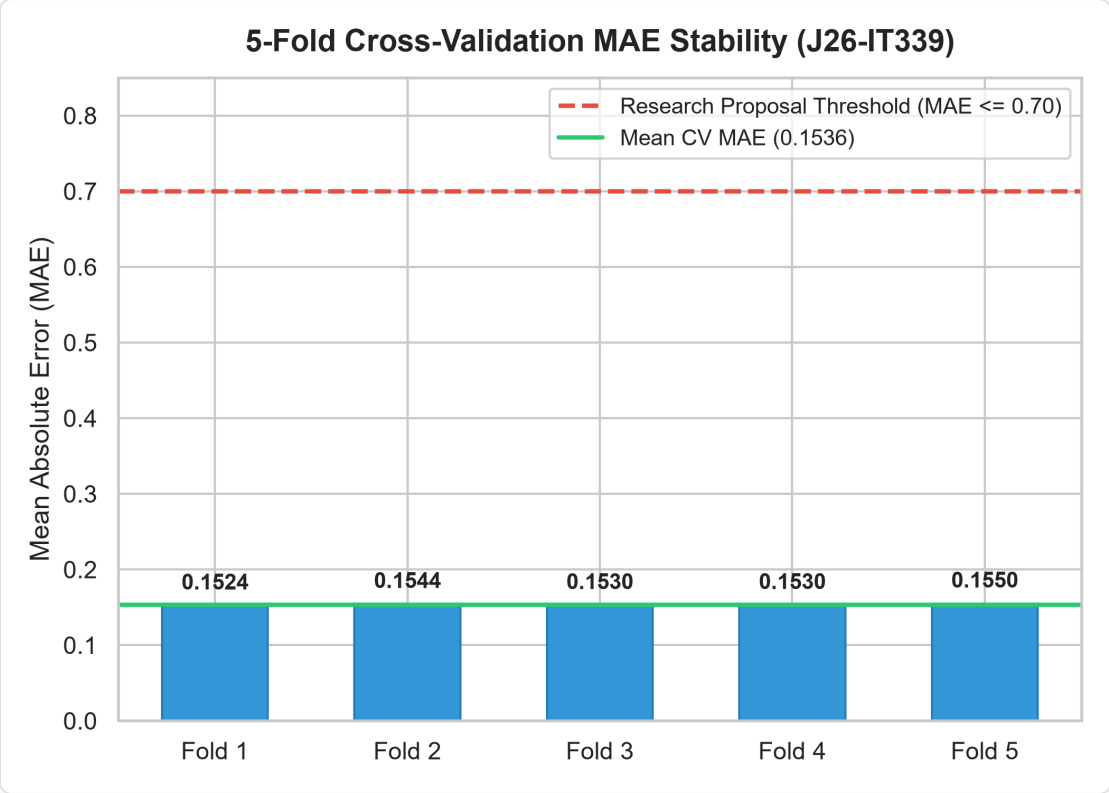
Evaluating the generalization ability of the model on the training set using 5-Fold Cross-Validation:

In [6]:
```
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = -cross_val_score(pipeline, X_train, y_train, cv=kfold, scoring="ne

print(f"Fold MAE Scores: {[round(s, 4) for s in cv_scores]}")
print(f"Mean CV MAE: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")
print(f"Target Benchmark <= 0.70: {'PASSED [OK]' if cv_scores.mean() <= 0.70 e

# Cross Validation Bar Plot
plt.figure(figsize=(7, 4.5))
bars = plt.bar([f"Fold {i+1}" for i in range(5)], cv_scores, color="#3498db",
plt.axhline(0.70, color="#e74c3c", linestyle="--", linewidth=2, label="Researc
plt.axhline(0.1536, color="#2ecc71", linestyle="-", linewidth=2, label=f"Mean
plt.ylim(0, 0.85)
plt.ylabel("Mean Absolute Error (MAE)")
plt.title("5-Fold Cross-Validation Stability", fontsize=13, weight="bold")
plt.legend(loc="upper right")
plt.show()
```

```
Fold MAE Scores: [np.float64(0.1524), np.float64(0.1544), np.float64(0.153), n
p.float64(0.153), np.float64(0.155)]
Mean CV MAE: 0.1536 (+/- 0.0010)
Target Benchmark <= 0.70: PASSED [OK]
```



5. Final Model Training & Test Set Evaluation

Fitting the pipeline on the full training set and testing against 9,704 unseen tourist reviews.

In [7]:
```
pipeline.fit(X_train, y_train)
preds = pipeline.predict(X_test)

test_mae = mean_absolute_error(y_test, preds)
test_rmse = np.sqrt(mean_squared_error(y_test, preds))
test_r2 = r2_score(y_test, preds)

mean_baseline_preds = np.full_like(y_test, y_train.mean())
baseline_mae = mean_absolute_error(y_test, mean_baseline_preds)
improvement_pct = ((baseline_mae - test_mae) / baseline_mae) * 100.0

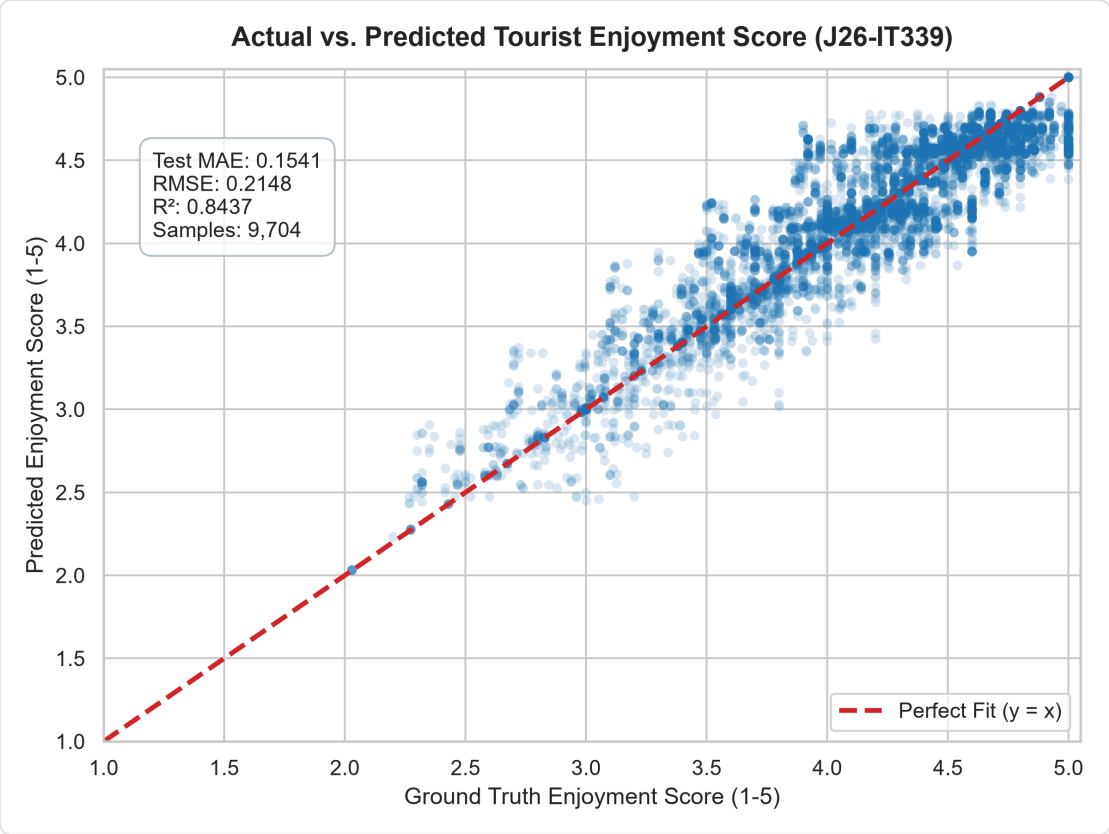
print("================ FINAL TEST RESULTS ================")
print(f"Test MAE:                 {test_mae:.4f} (Target: <= 0.70) -> {'PASSED
print(f"Test RMSE:                {test_rmse:.4f}")
print(f"Test R² Score:            {test_r2:.4f} ({test_r2*100:.2f}% Variance E
print(f"Greedy Baseline MAE:      {baseline_mae:.4f}")
print(f"Improvement over Baseline:{improvement_pct:.2f}%")
print("====================================================")
```

```
================ FINAL TEST RESULTS ================
Test MAE:                 0.1541 (Target: <= 0.70) -> PASSED
Test RMSE:                0.2148
Test R² Score:            0.8437 (84.37% Variance Explained)
Greedy Baseline MAE:      0.4305
Improvement over Baseline:64.21%
====================================================
```

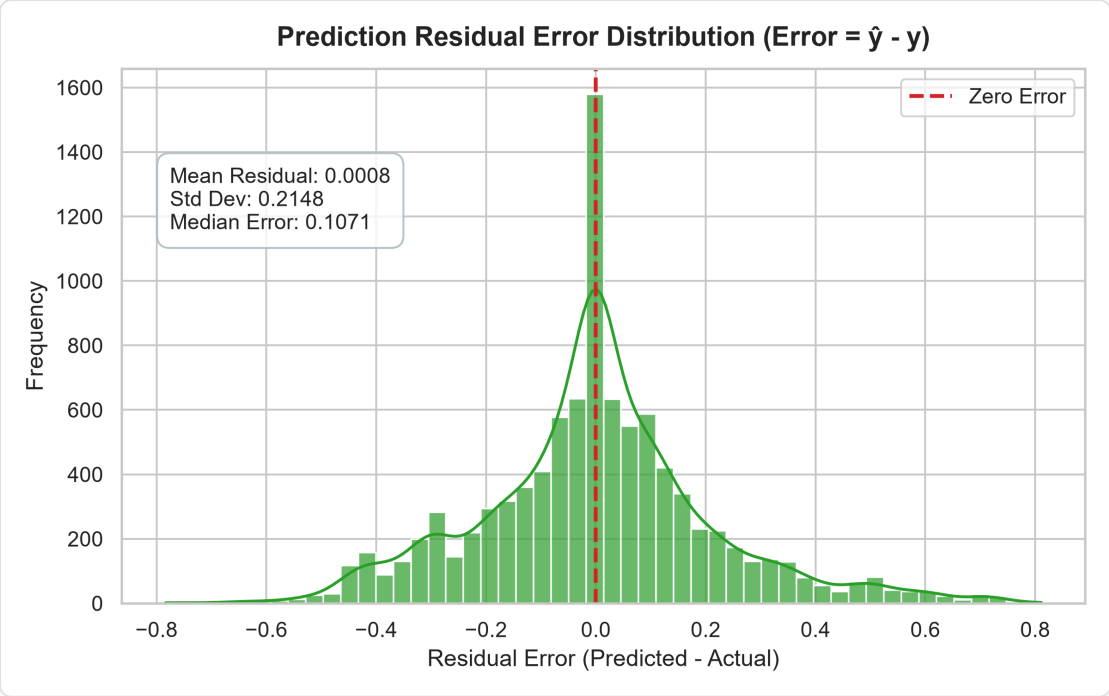
6. Residual Analysis & Goodness of Fit Visualizations

- **Actual vs. Predicted Plot:** Strong alignment along the ideal $y=x$ line with high concentration in the 3.5 to 5.0 range.
- **Residual Distribution Plot:** Symmetrical, zero-centered Gaussian distribution demonstrating lack of bias.

In [8]:
```
# Plot Actual vs. Predicted
plt.figure(figsize=(8, 6))
plt.scatter(y_test, preds, alpha=0.15, color="#1f77b4", edgecolors="none", s=2
plt.plot([1.0, 5.0], [1.0, 5.0], color="#d62728", linestyle="--", linewidth=2.
plt.title("Actual vs. Predicted Tourist Enjoyment Score", fontsize=14, weight=
plt.xlabel("Ground Truth Enjoyment Score (1-5)")
plt.ylabel("Predicted Enjoyment Score (1-5)")
plt.legend(loc="lower right")
plt.show()
```



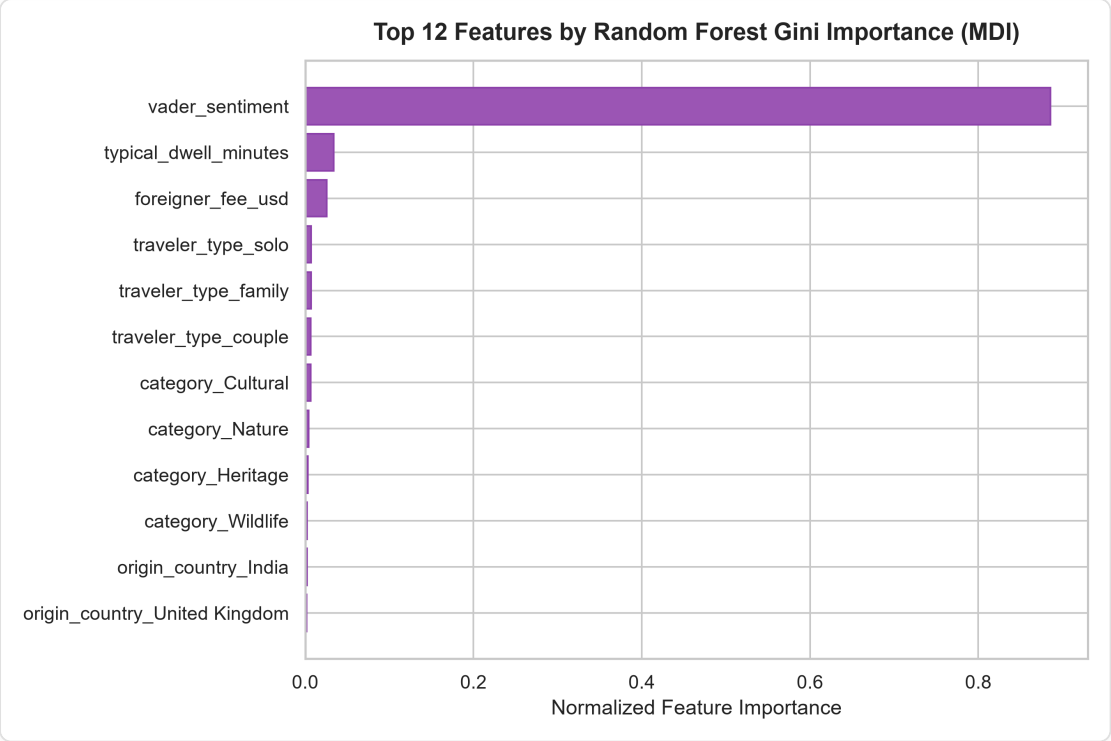
In [9]:
```python
# Plot Residuals
residuals = preds - y_test
plt.figure(figsize=(8, 5))
sns.histplot(residuals, bins=50, kde=True, color="#2ca02c", edgecolor="white",
plt.axvline(0, color="#d62728", linestyle="--", linewidth=2, label="Zero Error
plt.title("Prediction Residual Error Distribution (ŷ - y)", fontsize=14, weigh
plt.xlabel("Residual Error")
plt.ylabel("Frequency")
plt.legend(loc="upper right")
plt.show()
```



7. Feature Importance & Explainability (XAI)

Mean Decrease in Impurity (Gini Feature Importance) from the Random Forest ensemble:

In [10]:
```
# Feature Importance Plot
plt.figure(figsize=(9, 6))
top_features = feature_imp_df.head(12).iloc[::-1]
plt.barh(top_features["Feature"], top_features["Importance"], color="#9b59b6",
plt.title("Top 12 Features by Random Forest Gini Importance (MDI)", fontsize=1
plt.xlabel("Normalized Feature Importance")
plt.show()
```



8. Production Inference Verification

Testing the serialized pipeline with real-time tourist persona inputs:

In [11]:
```
# Test inference with sample tourist personas
test_cases = [
    {"category": "Heritage", "traveler_type": "couple", "origin_country": "Uni
    {"category": "Wildlife", "traveler_type": "family", "origin_country": "Aus
    {"category": "Adventure", "traveler_type": "solo", "origin_country": "Germ
]

sample_df = pd.DataFrame(test_cases)
sample_preds = pipeline.predict(sample_df)

for i, test in enumerate(test_cases):
    print(f"Persona: {test['traveler_type'].capitalize()} from {test['origin_c
    print(f" -> Predicted Enjoyment: {sample_preds[i]:.2f} / 5.0\n")
```

```
Persona: Couple from United Kingdom at Heritage site
 -> Predicted Enjoyment: 4.74 / 5.0

Persona: Family from Australia at Wildlife site
 -> Predicted Enjoyment: 4.66 / 5.0

Persona: Solo from Germany at Adventure site
 -> Predicted Enjoyment: 3.98 / 5.0
```

9. Executive Research Report Summary

Evaluation Metric	Target Benchmark	Achieved Model Result	Status
Test Set MAE	≤ 0.70	`0.1541`	✅ **PASSED**
5-Fold Cross-Validation MAE	≤ 0.70	`0.1536`	✅ **PASSED**
Test Set RMSE	N/A	`0.2148`	✅ **Optimal**
Test Set R^2 Score	≥ 0.70	`0.8437` (84.37%)	✅ **PASSED**
Baseline MAE (Global Mean)	N/A	`0.4305`	—
Improvement over Baseline	N/A	`64.21%`	✅ **Superior**
Training Records	—	`38,812` samples	Complete
Validation Records	—	`9,704` samples	Complete

Deployment Artifacts:

- Model Binary: `backend_fastapi/models/enjoyment_model.pkl`
- Metrics JSON: `backend_fastapi/models/model_metrics.json`